



File Validation Utility (FVU) Integration Document

Version 1.0

CENTRAL KYC RECORD REGISTRY (CKYCRR 2.0)

Version History

Version	Date	Details
V1.0	05/08/2026	FVU integration document prepared

Table of Contents

1. Overview	3
2. Prerequisites	4
3. Deployment Folder Layout	4
4. Configuration File Reference	5
5. Running the Command-Line File Validation Utility (SOP)	6
6. Output details.....	7
6.1. Exit Codes.....	7
6.2. Standard Output — JSON Summary.....	7
6.3. Output files.....	8
7. Integration Examples.....	8
8. Error handling	9

1. Overview

The File Validation Utility (FVU) is used to validate the input file along with its supporting documents and generate an output ZIP file containing the validated file containing hash value and FVU version.

The FVU is available in the following two variants:

- **Offline File Validation Utility (Manual Execution)**

The offline FVU can be downloaded and installed on the Reporting Entity's (RE) local system. It is intended for manual execution by the user and cannot be invoked programmatically by external applications.

- **Command-Line File Validation Utility (EXE)**

The Command-Line FVU is designed to support system-to-system integration and automation. It performs the same validation process as the offline FVU but can be invoked programmatically by external applications, scripts, schedulers, or automation platforms. The utility generates the validation output in the same ZIP format as the offline FVU.

Since the Command-Line FVU is provided as a standard executable (EXE), it can be integrated with applications developed in any programming language without requiring any SDK, library, or application-specific configuration.

Functional Processing

- Reads the YAML configuration file.
- Starts the backend Spring Boot JAR, if configured.
- Scans the configured input directory for supported file types (.SRC, .DWN, .UPL, .UPD, and .RECON.RE).
- Submits each input file to the REST-based validation API for validation.
- Generates the validated output files in the configured output directory.
- Creates an Executive Summary PDF containing the validation results.
- Writes execution logs (application.log and error.log) to the configured log directory.
- Prints a JSON summary of the validation results to the standard output (stdout).
- Returns an appropriate numeric exit code indicating the execution status.

2. Prerequisites

Before integrating and invoking the **Command-Line File Validation Utility (FVU)**, ensure that the following components are available and the required environment has been configured.

Component / Requirement	Description
FVU_RUN_UTILITY.exe	Executable provided by CERSAI. Copy the executable to the designated deployment directory.
config.yaml	Configuration file provided by CERSAI. Update the environment-specific parameters (such as folder paths, API URL, and other runtime settings) and place the file in the same directory as the executable.
lib\FVU-Utility.jar	Spring Boot backend JAR. Required only when backend auto-start is enabled in the config.yaml file.
jdk21\	Portable JDK 21 required to start the backend JAR. Applicable only when backend auto-start is enabled.
Input Folder	Ensure that the folder specified in the input_folder parameter of config.yaml exists and contains the supported input files for validation.
Network Connectivity	Ensure that the executing system has network connectivity to the Validation API endpoint specified in the api.base_url parameter of config.yaml.

3. Deployment Folder Layout

Deploy the **Command-Line File Validation Utility (FVU)** by placing the executable and all supporting files in a single deployment directory.

- Place **FVU_RUN_UTILITY.exe**, **config.yaml**, and all required supporting files in the same deployment directory.
- By default, the executable automatically reads the config.yaml file from the deployment directory. No command-line arguments are required for standard execution.

```
1.Deploy  FVU_RUN_UTILITY.exe  ← Executable file
2.config. yaml                 ← edit this file before execution
3.lib    FVU-Utility.jar       ← backend Spring Boot JAR
4.jdk21\                        ←portable JDK 21 (contains bin\java.exe)
```

Note: If the config.yaml file is located in a different directory, specify its complete path using the -c command-line option.

4. Configuration File Reference

The config.yaml file contains all configurable parameters required for executing the Command Line File Validation Utility. Before running the utility, update the configuration values according to the target environment using any text editor.

```
# Folder containing input files to validate
input_folder: D:/input

# Supporting documents folder (sent with each file)
doc_folder: D:/support_docs

# Output folder for reports and logs output_folder: D:/output

api:
    base_url: http://localhost:9091
    endpoint: /api/search-validate/file-with-support
    request_timeout_seconds: 600

backend: # remove section to skip auto-start
    jar_path: D:/deploy/lib/FVU-Utility.jar
    jdk_path: D:/deploy/jdk21
```

The following configuration parameters are supported:

Parameter	Required	Description
input_folder	Yes	Directory containing the input files to be validated. Supported file types include .SRC, .DWN, .UPL, .UPD, .DAC, and .RECON.RE.
output_folder	Yes	Directory where the generated validation output files and the Executive Summary PDF report will be stored.
log_folder	No	Directory where the application log files (application.log and error.log) will be generated. If not specified, the output_folder is used.

api.base_url	Yes	Base URL of the Validation API (for example, http://localhost:9091).
api.trigger_source	No	Trigger source value sent as part of the API request. The default value is BATCH .
api.request_timeout_seconds	No	Timeout, in seconds, for each validation request. The default value is 600 seconds .
backend.jar_path	No	Path to the FVU-Utility.jar file. Configure this parameter only when automatic startup of the backend service is required.
backend.jdk_path	No	Root directory of the Portable JDK 21 installation. The specified location must contain bin/java.exe. Required only when automatic backend startup is enabled.
backend.jvm_args	No	Additional JVM arguments to be passed while starting the backend service (for example, -Xmx512m).

5. Running the Command-Line File Validation Utility (SOP)

The following procedure describes the steps to execute the **Command-Line File Validation Utility (FVU)**. Before proceeding, ensure that all prerequisites have been completed and the config.yaml file has been configured with the appropriate environment-specific values.

Execution Steps

Step 1: Copy the FVU_RUN_UTILITY.exe, config.yaml, lib folder, and jdk21 folder (if applicable) to the deployment directory.

Step 2: Update the configuration parameters in the config.yaml file, such as the input folder, output folder, log folder, and Validation API URL.

Step 3: Place the input files (.SRC, .DWN, .UPL, .UPD, .DAC, .RECON.RE) in the configured input_folder.

Step 4: Invoke the FVU_RUN_UTILITY.exe from the integrating application by passing the path to the config.yaml file (if required). The application shall execute the EXE and wait for the execution to complete.

Step 5: Verify the execution status by checking the returned exit code and reviewing the JSON response displayed in the console.

Step 6: Review the generated output files, including the Executive Summary PDF, application.log, and error.log, in the configured output and log directories.

6. Output details

6.1. Exit Codes

After the execution is complete, the Command-Line File Validation Utility (FVU) terminates and returns one of the following exit codes to indicate the execution status.

Exit Code	Description	Recommended Action
0	All input files were validated successfully.	The validation process completed successfully. No further action is required.
2	Configuration error (for example, invalid config.yaml, missing configuration parameter, or invalid folder path).	Verify the configuration in config.yaml, correct the error, and re-run the utility.
3	One or more input files failed validation.	Review error.log and the Executive Summary PDF report for detailed validation errors.
1	An unexpected fatal error occurred during execution.	Review the standard error (stderr) output for the exception details and resolve the reported issue.

6.2. Standard Output — JSON Summary

Upon successful execution (Exit Code 0 or 3), the Command-Line File Validation Utility (FVU) prints a single JSON object to the standard output (stdout). The invoking application can parse this JSON response to retrieve the validation summary, including the number of files processed and the location of the generated Executive Summary PDF report.

Field Name	Data Type	Description
totalFiles	Integer	Total number of files found in the input folder.
success	Integer	Number of files that passed validation.
failed	Integer	Number of files that failed validation.
summaryPdf	String	Absolute path to the Executive Summary PDF report.


```
{
  "totalFiles" : 5,
  "success"    : 4,
  "failed"     : 1,
  "summaryPdf" : "D:/output/Executive_Summary_20260629_223526.pdf"
}
```

6.3. Output files

After the validation process is completed, the utility generates the following output files.

File	Location	Description
Output ZIP file along with Executive_Summary_*.pdf	output_folder	Output zip file along with executive Summary report containing file-wise validation results and overall statistics.
application.log	log_folder	Detailed application log containing execution and processing information.
error.log	log_folder	Log containing validation and execution errors for troubleshooting

7. Integration Examples

The following examples demonstrate how to invoke the Command-Line File Validation Utility from different programming languages. Regardless of the programming language or technology stack used, the integration process remains the same.

- Invoke the Command-Line File Validation Utility by launching the executable as a child process.
- Capture the standard output (stdout) and parse the JSON response returned by the utility.
- Verify the exit code returned by the utility and perform the appropriate action based on the execution status.

8. Error handling

The following table lists the common errors that may occur during the execution of the Command-Line File Validation Utility, along with their possible causes and the recommended resolution.

Error code	Error cause	Resolution
Exit code 2: "Config file not found"	The utility is unable to locate the config.yaml file.	Specify the complete path to the configuration file using the -c option or place the config.yaml file in the same directory as the executable.
Exit code 2: "input_folder does not exist"	The configured input_folder path is invalid or the directory does not exist.	Create the folder or fix the path in config.yaml
Exit code 1: "Backend JAR not found"	jar_path in config.yaml points to a location where the JAR does not exist.	Use an absolute path to FVU-Utility.jar , or remove the backend section to skip auto-start.
Exit code 1: JDK not found / java.exe missing	jdk_path does not point to a valid JDK 21 installation.	Verify that jdk_path in \java.exe exists.
Exit code 3 for every file	The backend API is not reachable or returning validation errors.	Check api.base_url in config.yaml , verify the backend is running, and review application.log .
stdout is empty / JSON parse fails	The EXE failed before printing output (exit code 1 or 2).	Read stderr for the error message.
EXE closes instantly with no output	The EXE was double-clicked — it is a console app.	Run from a terminal / command prompt or call from code that captures output.